

Introduction to Applied Cryptography – LDACS1310

Exercises Session 3

Exercise #1. The first task in this exercise session is to implement the cipher against which we will perform cryptanalysis. For simplicity, we chose to use a toy cipher based on a real (standardized) lightweight cipher, namely PRESENT. This toy cipher is represented at the end of the document with four rounds.

The toy cipher is based on a Substitution-Permutation Network (SPN) structure. The SPN design iterates non-linear operations (i.e. Substitution-boxes) and linear ones (i.e. bit-permutations), interleaved with key additions (i.e. bitwise XORs). It takes as inputs a plaintext and key of 32 bits and outputs a 32 bits ciphertext. No key schedule is performed: the key used is the same for each round. You may have noticed that an additional key addition is performed at the end of the cipher

We provide in `utils.py`, the functions `sbox()` and `permutation_layer()` to perform the substitution and permutation layers along with their inverse `sbox_inv()` and `permutation_layer_inv()`.

Your goal is to implement the toy cipher encryption and decryption using these tables. Your code should pass the tests in the main of `e1.py`. To implement the encryption and decryption functions.

Exercise #2. To attack the toy cipher using linear cryptanalysis, we need to identify a linear relation between some plaintext bits and ciphertext bits that occurs with a probability different from $1/2$. The S-box is a non-linear component of the cipher. Therefore, an attacker cannot simply write down an exact linear relation between the input and output bits. Instead, he is looking for linear approximations of the S-box that would lead to linear relation holding more or less often than expected by chance.

Consider an S-box

$$S : \{0, 1\}^n \rightarrow \{0, 1\}^n.$$

For an input mask $\alpha \in \{0, 1\}^n$ and an output mask $\beta \in \{0, 1\}^n$, we consider the linear relation

$$\alpha \cdot x = \beta \cdot S(x) \tag{1}$$

where $\cdot$ denotes the bitwise inner product over $\mathbb{F}_2$.

If the S-box behaved like a random bijection, we would expect that eq. (1) holds for approximately half of all possible inputs. A deviation from this probability is called a *bias*.

More precisely, for a given pair of masks (α, β) , let

$$N_{\alpha, \beta} = \# \{x \in \{0, 1\}^n : \alpha \cdot x = \beta \cdot S(x)\}.$$

be the number of values of x for which eq. (1) holds. The bias of the corresponding linear approximation is defined as

$$\varepsilon_{\alpha, \beta} = \frac{N_{\alpha, \beta}}{2^n} - \frac{1}{2}.$$

The goal of this exercise is to compute the *bias table* of the S-box.

1. For each pair (α, β) , evaluate the linear relation

$$\alpha \cdot x = \beta \cdot S(x)$$

for every possible input x .

2. Count the number $N_{\alpha, \beta}$ of inputs for which the relation holds.
3. Compute the corresponding bias

$$\varepsilon_{\alpha, \beta} = \frac{N_{\alpha, \beta}}{2^n} - \frac{1}{2}$$

4. Fill in the complete bias table and identify the input/output mask pairs with the largest absolute bias.

These biased linear approximations are the building blocks of a linear cryptanalysis attack. In the following exercises, we will use them to construct a linear approximation through several components of the toy cipher and exploit the resulting bias to recover information about the key.

Exercise #3. In a linear cryptanalysis attack, the attacker searches for a linear approximation that relates some input bits of the cipher to some output bits after R rounds. The objective is to find an approximation whose probability differs from $1/2$, i.e., whose bias is non-zero.

When an approximation passes through several S-boxes, the biases of the individual S-box approximations can be combined using the *piling-up lemma*. For two independent approximations with biases ε_1 and ε_2 , the bias of their combination is $\varepsilon_{\text{tot}} = 2\varepsilon_1\varepsilon_2$. More generally, when n independent approximations are combined, the resulting bias is

$$\varepsilon_{\text{tot}} = 2^{n-1} \prod_{i=1}^n \varepsilon_i$$

Consider the toy cipher shown at the end of the document. We want to construct a two-round linear approximation in which only the S-box S_5 of the first round and the S-box S_2 of the second round are active. Assume that the input mask of the first active S-box is $\alpha_1 = 0\text{x}8$ and the output mask of the second active S-box is $\beta_2 = 0\text{x}8$.

1. Using the representation of the toy cipher and the bias table computed in Exercise #2, draw the unique two-round linear approximation having the constraints stated above and find β_1 and α_2 .
2. Find theoretical bias of the complete two-round approximation using the piling-up lemma:

$$\varepsilon_{\text{tot}} = 2 \cdot \varepsilon_{\alpha_1, \beta_1} \cdot \varepsilon_{\alpha_2, \beta_2}.$$

3. We now want to verify the theoretical bias experimentally. Using `e3.py`, generate random plaintext/ciphertext pairs and evaluate the empirical bias of the two rounds linear approximation.
 - (a) Why should the key remain fixed during the experiment?
 - (b) How does the empirical bias evolve as the number of plaintext/ciphertext pairs increases?
 - (c) How many samples are required to obtain a sufficiently stable bias estimate ? Can you relate this to any formula ?
 - (d) Compare the empirical bias with the theoretical one obtained using the piling-up lemma. Explain any difference between the two values.
 - (e) Why the sign of the ε_{tot} changes between executions ?

Exercise #4. Once the attacker has found a linear approximation on R rounds with a non-zero bias, this approximation can be used to recover information about the key of the $R + 1^{\text{th}}$ round.

To identify the key bits, the attacker extends the active path of the linear approximation through the $R + 1^{\text{th}}$ round. The S-boxes that are active in the last round determine which key bits are involved in the attack.

Consider the linear approximation obtained in of Exercise #3.

1. Extend the active path through the third round and identify the active S-boxes. Which key bits are targeted by the attack ?
2. Draw the complete active path on the toy cipher representation.

The attacker can recover the targeted key bits by using a large number of plaintext/ciphertext pairs. For each key guess, the attacker use that guess to partially reverse the last round of all plaintext/ciphertext pairs. He can then evaluate the linear approximation of Exercise #3 and compute the empirical bias.

3. What is the expected emperical bias for the correct key values ? And for a wrong key guess ?

Complete `e4.py` to perform the attack.

Exercise #5. A set of plaintext/ciphertext pairs, encrypted under a randomly chosen key, is provided in the files `plaintexts.npy` and `ciphertexts.npy`. Your goal is to use linear cryptanalysis to recover the secret key. Using the bias table from Exercise #2, identify several two-round linear approximations with a non-zero bias. Edit `e5.py` to implement your attack and check your key guess against the actual key used.



